

Problem_Set_2

Group 2

2026-02-26

1.) Use the bisection method with a hand calculator or computer to find the real root of $x^3 - x - 3 = 0$. Use an error tolerance of $\epsilon = 0.0001$. Graph the function $f(x) = x^3 - x - 3$ and label the root.

Solution:

Finding the root using bisection, given $[a, b] = [1, 2]$.

```
library(cmna)
f_x <- function(x){x^3-x-3}

root <- bisection(f_x, 1, 2, tol=0.0001)
root
```

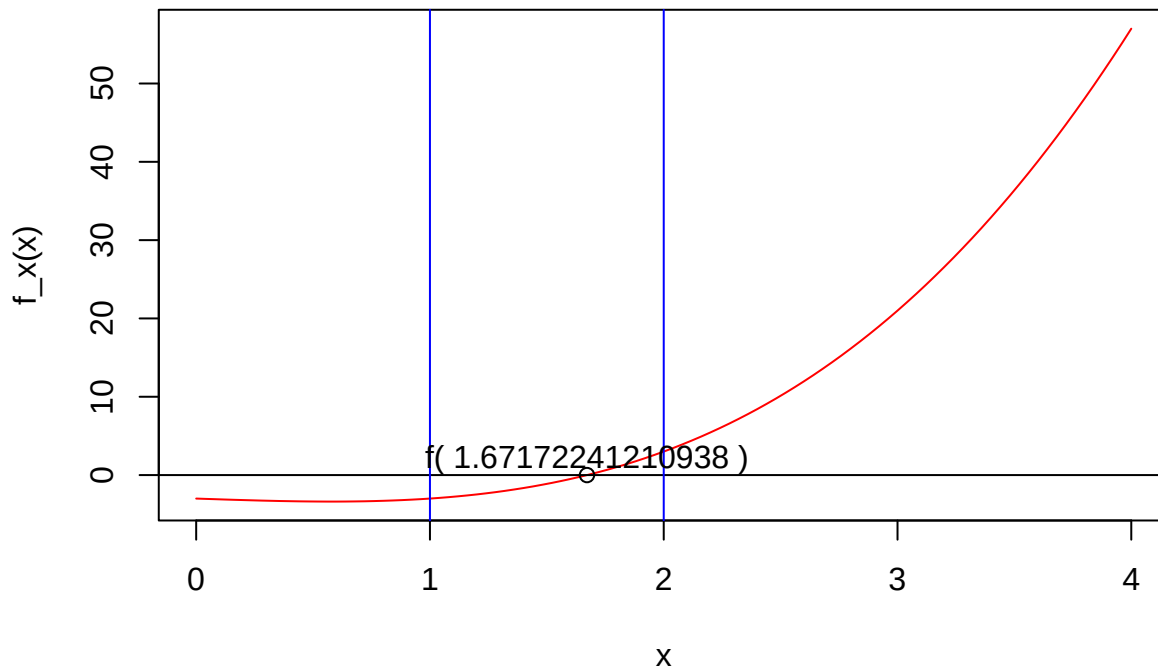
```
## [1] 1.671722
```

The root is 1.671722

```
curve(f_x, from=0, to=4, col='red')
abline(h=0)

# a and b
abline(v=1, col='blue')
abline(v=2, col='blue')

# root
points(root, 0)
text(root, 2, paste('f(',root,')'))
```



2.) The function $f(x) = -3x^3 + 2e^{\frac{x^2}{2}} - 1$ has values of zero near $x = -0.5$ and $x = 0.5$.

a. What is the derivative of f ?

Answer:

$$\begin{aligned} &= -3x^3 + 2e^{\frac{x^2}{2}} - 1 \\ &= -9x^2 + 2xe^{\frac{x^2}{2}} \\ &= x(2e^{\frac{x^2}{2}} - 9x) \end{aligned}$$

$$x(2e^{\frac{x^2}{2}} - 9x)$$

b. If you begin Newton's method at $x = 1$, which root is reached? How many iterations to achieve an error less than 10^{-5} ? Answer:

```
f_x <- function(x) { -3*x^3 + 2 * exp(x^2/2) - 1 }
f_prime <- function(x) { 2 * x * exp(x^2 / 2) - 9 * x^2 }
```

```
MyNewton <- function(func, d_func, x, eps){
  print(x)
  x_next = x - (func(x) / d_func(x))
  if (abs(x_next - x) <= eps){
    return(x_next)
  }

  return(MyNewton(func, d_func, x_next, eps))
}
```

```
MyNewton(f_x, f_prime, 1, 0.00001)
```

```
## [1] 1
## [1] 0.8767996
## [1] 0.8572855
## [1] 0.8568009
```

```
## [1] 0.8568006
```

Therefore, the root is 0.8568006 when $x = 1$

c. Begin Newton's method at another starting point to get the other zero.

Answer: Using the starting point $x = 2$

```
MyNewton(f_x, f_prime, 2, 0.00001)
```

```
## [1] 2
## [1] 0.4136804
## [1] 1.926073
## [1] 0.8251612
## [1] 0.8581445
## [1] 0.8568029
## [1] 0.8568006
```

Therefore, the root is 0.8568006 when $x = 2$

3.) Use the function from no. 2 and find the root using the secant method where $x_0 = 0$ and $x_1 = 1$. Use an error tolerance of $\epsilon = 0.001$.

```
MySecant <- function(f, x_prev, x, tol){
  num <- x - x_prev
  den <- f(x) - f(x_prev)

  next_x <- x - f(x) * (num / den)

  if(abs(next_x - x) <= tol){
    return(next_x)
  }

  print(next_x)
  return(MySecant(f, x, next_x, tol))
}
```

```
MySecant(f_x, 0, 1, 0.001)
```

```
## [1] 0.5873517
## [1] 0.8029441
## [1] 0.882793
## [1] 0.8549891
## [1] 0.8567427
## [1] 0.8568007
```

Therefore, the root is 0.8568007 when $x_0 = 0, x_1 = 1$, and $\epsilon = 0.001$ using the Secant Method

4.) Consider the system of equations:

$$\begin{cases} 2x + 3y - z = 5 \\ 4x - y + 2z = 6 \\ -3x + 2y + z = -4 \end{cases}$$

a. Present the augmented matrix of the system. Answer:

$$\left(\begin{array}{ccc|c} 2 & 3 & -1 & 5 \\ 4 & -1 & 2 & 6 \\ -3 & 2 & 1 & -4 \end{array} \right)$$

b. Solve the system using $Ax = LUx = Ly = b$ and round the final answer to 4 decimal digits.

```
LU <- function(mat){
  upper <- mat
  n <- nrow(mat)
  lower <- diag(n)

  for(col in 1:(n-1)){
    for(row in (col+1):n){
      m <- upper[row, col] / upper[col, col]
      lower[row, col] <- m
      upper[row,] = upper[row,] - m * upper[col,]
      print(sprintf("Row Operation: R_%d ← R_%d - (%.2f)R_%d",
                    row, row, m, col))
      print("Upper:")
      print(upper)
      print("Lower:")
      print(lower)
    }
  }

  return(list(L = lower, U = upper))
}

A = matrix(c(2, 3, -1, 4, -1, 2, -3, 2, 1), nrow=3, byrow=TRUE)
b = matrix(c(5, 6, -4), nrow=3, byrow=TRUE)
lu_results = LU(A)
```

```
## [1] "Row Operation: R_2 ← R_2 - (2.00)R_1"
## [1] "Upper:"
##      [,1] [,2] [,3]
## [1,]    2    3   -1
## [2,]    0   -7    4
## [3,]   -3    2    1
## [1] "Lower:"
##      [,1] [,2] [,3]
## [1,]    1    0    0
## [2,]    2    1    0
## [3,]    0    0    1
## [1] "Row Operation: R_3 ← R_3 - (-1.50)R_1"
## [1] "Upper:"
##      [,1] [,2] [,3]
## [1,]    2  3.0 -1.0
## [2,]    0 -7.0  4.0
## [3,]    0  6.5 -0.5
## [1] "Lower:"
##      [,1] [,2] [,3]
## [1,]  1.0    0    0
## [2,]  2.0    1    0
## [3,] -1.5    0    1
```

```
## [1] "Row Operation: R_3 ← R_3 - (-0.93)R_2"
## [1] "Upper:"
##      [,1] [,2]      [,3]
## [1,]    2    3 -1.000000
## [2,]    0   -7  4.000000
## [3,]    0    0  3.214286
## [1] "Lower:"
##      [,1]      [,2] [,3]
## [1,]  1.0  0.0000000  0
## [2,]  2.0  1.0000000  0
## [3,] -1.5 -0.9285714  1
```

```
L = lu_results$L
U = lu_results$U
```

```
# Solving for y using Ly = b
n <- nrow(L)
y <- matrix(0, nrow=3, ncol=1)
```

```
for(i in 1:n){
  sum_l <- 0
  if(i > 1){
    for(j in 1:(i-1)){
      sum_l <- sum_l + L[i,j] * y[j,1]
    }
  }
  y[i,1] <- b[i,1] - sum_l
}
print(round(y, 4))
```

```
##      [,1]
## [1,]  5.0000
## [2,] -4.0000
## [3,] -0.2143
```

```
# Solving for x using Ux = y
x = matrix(0, nrow=3, ncol=1)
```

```
for(i in 3:1){
  sum_u <- 0
  if (i < n) {
    for(j in (i+1):n){
      sum_u <- sum_u + U[i,j] * x[j,1]
    }
  }
  x[i,1] <- (y[i,1] - sum_u) / U[i,i]
}
print(round(x, 4))
```

```
##      [,1]
## [1,]  1.6667
## [2,]  0.5333
## [3,] -0.0667
```

Therefore, the final answer is $x = 1.6667$, $y = 0.5333$, $z = -0.0667$

c. Find the residual vector if the correct solution is: $x = 1, y = 2, z = -1$ Answer:

The residual vector can be solved by finding the difference between the correct solution and the answers from the previous question:

$$\begin{aligned}x_{\text{residual}} &= 1 - 1.6667 = -0.6667 \\y_{\text{residual}} &= 2 - 0.5333 = 1.4667 \\z_{\text{residual}} &= -1 - (-0.0667) = -0.9333\end{aligned}$$

5.) Compute the Frobenius norm, maximum column sum, and maximum row sum of the matrix:

$$\begin{pmatrix} 10.2 & 2.4 & 4.5 \\ -2.3 & 7.7 & 11.1 \\ -5.5 & -3.2 & 0.9 \end{pmatrix}$$

Answer:

```
A <- matrix(c(10.2, -2.3, -5.5, 2.4, 7.7, -3.2, 4.5, 11.1, 0.9), nrow=3, byrow=TRUE)
A_2 <- A^2
n <- nrow(A_2)
sum <- 0

col_norms <- c()
row_norms <- c()

for(i in 1:n){
  col_sum <- 0
  row_sum <- 0
  for(j in 1:n){
    sum = sum + A_2[i,j]
    col_sum <- col_sum + abs(A[i,j])
    row_sum <- row_sum + abs(A[j,i])
  }
  col_norms <- append(col_norms, col_sum)
  row_norms <- append(row_norms, row_sum)
}

# Frobenius Norm
sum

## [1] 359.14
sqrt(sum)

## [1] 18.95099
# Maximum Column
col_norms

## [1] 18.0 13.3 16.5
max(col_norms)

## [1] 18
# Maximum Row
row_norms
```

```
## [1] 17.1 21.1 9.6
```

```
max(row_norms)
```

```
## [1] 21.1
```

6.) Solve the system of equations given in no.4, starting with the initial vector of [0,0,0]:

a. Solve using the Jacobi method with 2-digit precision.

```
A <- matrix(c(2, 3, -1, 4, -1, 2, -3, 2, 1), nrow=3, byrow=TRUE)
lu_results <- LU(A)
```

```
## [1] "Row Operation: R_2 ← R_2 - (2.00)R_1"
```

```
## [1] "Upper:"
```

```
##      [,1] [,2] [,3]
```

```
## [1,] 2 3 -1
```

```
## [2,] 0 -7 4
```

```
## [3,] -3 2 1
```

```
## [1] "Lower:"
```

```
##      [,1] [,2] [,3]
```

```
## [1,] 1 0 0
```

```
## [2,] 2 1 0
```

```
## [3,] 0 0 1
```

```
## [1] "Row Operation: R_3 ← R_3 - (-1.50)R_1"
```

```
## [1] "Upper:"
```

```
##      [,1] [,2] [,3]
```

```
## [1,] 2 3.0 -1.0
```

```
## [2,] 0 -7.0 4.0
```

```
## [3,] 0 6.5 -0.5
```

```
## [1] "Lower:"
```

```
##      [,1] [,2] [,3]
```

```
## [1,] 1.0 0 0
```

```
## [2,] 2.0 1 0
```

```
## [3,] -1.5 0 1
```

```
## [1] "Row Operation: R_3 ← R_3 - (-0.93)R_2"
```

```
## [1] "Upper:"
```

```
##      [,1] [,2] [,3]
```

```
## [1,] 2 3 -1.000000
```

```
## [2,] 0 -7 4.000000
```

```
## [3,] 0 0 3.214286
```

```
## [1] "Lower:"
```

```
##      [,1] [,2] [,3]
```

```
## [1,] 1.0 0.0000000 0
```

```
## [2,] 2.0 1.0000000 0
```

```
## [3,] -1.5 -0.9285714 1
```

```
L <- lu_results$L
```

```
U <- lu_results$U
```

```
diag_gen <- function(mat){
```

```
  n <- nrow(mat)
```

```
  diag <- matrix(0, nrow=n, ncol=n)
```

```
  for(i in 1:n){
```

```
    diag[i,i] <- mat[i,i]
```

```
  }
```

```
  return(diag)
```

```

}

x = matrix(0, nrow=3, ncol=1)
b = matrix(c(5,6,-4), nrow=3, byrow=TRUE)
D <- diag_gen(A)

```

Matrix A

A

```

##      [,1] [,2] [,3]
## [1,]    2    3   -1
## [2,]    4   -1    2
## [3,]   -3    2    1

```

b

b

```

##      [,1]
## [1,]    5
## [2,]    6
## [3,]   -4

```

Diagonal

D

```

##      [,1] [,2] [,3]
## [1,]    2    0    0
## [2,]    0   -1    0
## [3,]    0    0    1

```

Lower triangle

L

```

##      [,1]      [,2] [,3]
## [1,]  1.0  0.0000000  0
## [2,]  2.0  1.0000000  0
## [3,] -1.5 -0.9285714  1

```

Upper Triangle

U

```

##      [,1] [,2]      [,3]
## [1,]    2    3 -1.000000
## [2,]    0   -7  4.000000
## [3,]    0    0  3.214286

```

b. Solve using Gauss-Seidel method with 2-digit precision.

c. Solve for $\bar{e}$ if the true solution is: $x = 1, y = 2, z = -1$